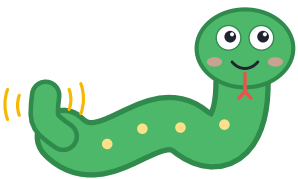
Lecture 8

Part 3: Functions and Dictionaries

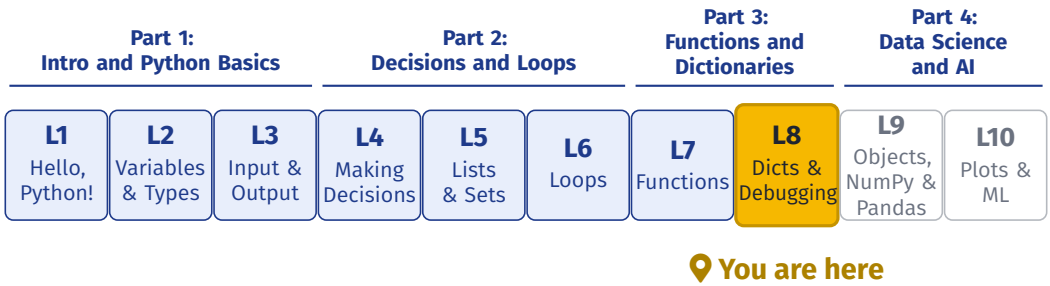
Dictionaries & Debugging

Look things up by name, work with tables, fix and catch bugs, and code with AI



Dror Jacoby
Tel Aviv University

Where we are


Part 1:
Intro and Python Basics
Part 2:
Decisions and Loops
Part 3:
Functions and
Dictionaries
Part 4:
Data Science
and AI
L1
Hello,
Python!
L2
Variables
& Types
L3
Input &
Output
L4
Making
Decisions
L5
Lists
& Sets
L6
Loops
L7
Functions
L8
Dicts &
Debugging
L9
Objects,
NumPy &
Pandas
L10
Plots &
ML
You are here

Quick: what is Noa's phone number?

- Your phone holds hundreds of contacts
- You never ask for "contact number 217"

Quick: what is Noa's phone number?

- Your phone holds hundreds of contacts
- You never ask for "contact number 217"
- You type a **name** and get the number
- Today Python learns to look things up by name





Today's plan

◎ By the end of today you can…

- Store, look up and count things with dictionaries
- Work with tables and find the median of a list
- Read error messages, hunt bugs, catch errors with `try` / `except`, and use AI wisely



Today's outline

1 Dictionaries
Look up a value by its key: create, change, `in`, `get`, count

2 Tables and the middle value
Lists of lists, lists of dictionaries, median and quantiles by hand

3 Debugging
Read the error from the bottom up, the common errors, logic bugs, catching errors with `try` / `except`

4 Coding with AI
AI can be confidently wrong: good prompts and testing what it writes



Every topic: examples, **Try it!** and **Predict the output**. At the end: **Bug Zoo**, **AI Corner**, **Recap**.

Why today matters

Why learn this?

- Most real data is “name → value”: prices, grades, phone numbers
- Programmers spend much of their time debugging. Do it with a method. Plan for errors with `try`
- AI writes code fast. Only you can check it is right

Where you will see it

- Your phone’s contacts: name → number
- Data from websites (JSON) is dictionaries
- House prices and salaries are reported as medians





Topic 1

Dictionaries

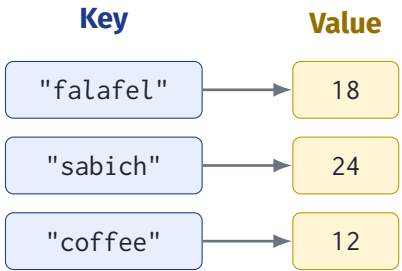
Lecture 8 • Dicts & Debugging

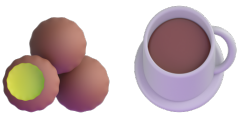


TODAY

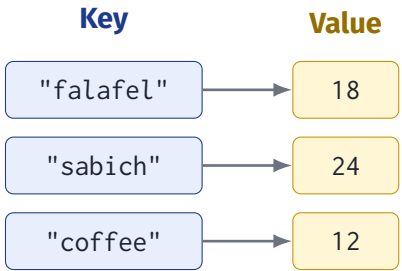
1. **Dictionaries**
2. Tables and the middle value
3. Debugging
4. Coding with AI

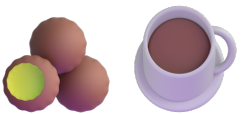
A dictionary: keys and values





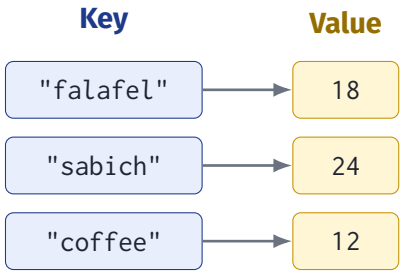
A dictionary: keys and values

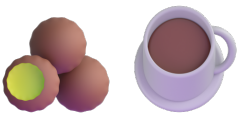




- Like a menu: **key** = dish, **value** = price

A dictionary: keys and values





- Like a menu: **key** = dish, **value** = price
- Each key appears only once: one price per dish

Create, look up, change

L08_01_dictionaries.ipynb

```
menu = {"falafel": 18, "sabich": 24, "coffee": 12}
print(menu["sabich"])
menu["shakshuka"] = 38    # new key: add
menu["coffee"] = 14       # old key: update
print(menu)
```

Create, look up, change

L08_01_dictionaries.ipynb

```
menu = {"falafel": 18, "sabich": 24, "coffee": 12}
print(menu["sabich"])
menu["shakshuka"] = 38    # new key: add
menu["coffee"] = 14       # old key: update
print(menu)
```

OUTPUT

```
24
{'falafel': 18, 'sabich': 24, 'coffee': 14, 'shakshuka': 38}
```

Create, look up, change

L08_01_dictionaries.ipynb

```
menu = {"falafel": 18, "sabich": 24, "coffee": 12}
print(menu["sabich"])
menu["shakshuka"] = 38    # new key: add
menu["coffee"] = 14       # old key: update
print(menu)
```

OUTPUT

```
24
{'falafel': 18, 'sabich': 24, 'coffee': 14, 'shakshuka': 38}
```

Curly braces `{ }` build it. Square brackets with a key look it up.

Is it there? `in` and `get`

```
phones = {"Noa": "050-1234567",
          "Omer": "052-7654321"}
print("Noa" in phones)
print("Yossi" in phones)
print(phones.get("Noa"))
print(phones.get("Yossi", "not saved"))
```

Is it there? `in` and `get`

```
phones = {"Noa": "050-1234567",
          "Omer": "052-7654321"}
print("Noa" in phones)
print("Yossi" in phones)
print(phones.get("Noa"))
print(phones.get("Yossi", "not saved"))
```

OUTPUT

```
True
False
050-1234567
not saved
```

Is it there? `in` and `get`

```
phones = {"Noa": "050-1234567",
          "Omer": "052-7654321"}
print("Noa" in phones)
print("Yossi" in phones)
print(phones.get("Noa"))
print(phones.get("Yossi", "not saved"))
```

OUTPUT

```
True
False
050-1234567
not saved
```

- `in` checks the **keys**
- `get` gives a backup answer instead of crashing

Looping over a dictionary

```
menu = {"falafel": 18,
        "sabich": 24,
        "coffee": 12}
for item, price in menu.items():
    print(f"{item}: {price} ILS")
print(sum(menu.values()))
```

Looping over a dictionary

```
menu = {"falafel": 18,
        "sabich": 24,
        "coffee": 12}
for item, price in menu.items():
    print(f"{item}: {price} ILS")
print(sum(menu.values()))
```

OUTPUT

```
falafel: 18 ILS
sabich: 24 ILS
coffee: 12 ILS
54
```

Looping over a dictionary

```python
menu = {"falafel": 18,
        "sabich": 24,
        "coffee": 12}
for item, price in menu.items():
    print(f"{item}: {price} ILS")
print(sum(menu.values()))
```

OUTPUT

```
falafel: 18 ILS
sabich: 24 ILS
coffee: 12 ILS
54
```

- `.items()`: each key **with** its value
- `.keys()`: only keys. `.values()`: only values

Counting votes: a tally board

```
votes = ["pizza", "sushi", "pizza", "falafel", "pizza", "sushi"]
counts = {}
for food in votes:
    if food in counts:
        counts[food] = counts[food] + 1    # seen it: add 1
    else:
        counts[food] = 1                   # new: start at 1
print(counts)
```

pizza |||
sushi ||
falafel |



Counting votes: a tally board

```
votes = ["pizza", "sushi", "pizza", "falafel", "pizza", "sushi"]
counts = {}
for food in votes:
    if food in counts:
        counts[food] = counts[food] + 1    # seen it: add 1
    else:
        counts[food] = 1                   # new: start at 1
print(counts)
```

OUTPUT

```
{'pizza': 3, 'sushi': 2, 'falafel': 1}
```

pizza |||
sushi ||
falafel |



A shorter counter with `get`

```
reactions = ["laugh", "like", "laugh", "clap", "laugh", "like"]
counts = {}
for r in reactions:
    counts[r] = counts.get(r, 0) + 1
print(counts)
```



A shorter counter with `get`

```
reactions = ["laugh", "like", "laugh", "clap", "laugh", "like"]
counts = {}
for r in reactions:
    counts[r] = counts.get(r, 0) + 1
print(counts)
```

OUTPUT

```
{'laugh': 3, 'like': 2, 'clap': 1}
```



A shorter counter with `get`

```
reactions = ["laugh", "like", "laugh", "clap", "laugh", "like"]
counts = {}
for r in reactions:
    counts[r] = counts.get(r, 0) + 1
print(counts)
```

OUTPUT

```
{'laugh': 3, 'like': 2, 'clap': 1}
```

`get(r, 0)`: the count so far, or 0 if new.



Try it!

Try it in Colab notebooks/L08/L08_01_dictionaries.ipynb

1. Make a dictionary: 3 songs → length in minutes
2. Add a song, then change one length
3. Count the letters of your name with `get`





Predict the output

Predict the output!

```
counts = {}
for letter in "banana":
    counts[letter] = counts.get(letter, 0) + 1
print(counts)
```



Predict the output

Predict the output!

```
counts = {}
for letter in "banana":
    counts[letter] = counts.get(letter, 0) + 1
print(counts)
```



Answer

`{'b': 1, 'a': 3, 'n': 2}` Keys keep the order they were first seen.

Topic 2

Tables and the middle value

Lecture 8 • Dicts & Debugging



TODAY

1 Dictionaries

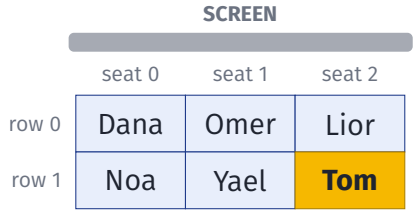
2 **Tables and the middle value**

3 Debugging

4 Coding with AI

A table is a list of rows

```
seats = [["Dana", "Omer", "Lior"],
         ["Noa", "Yael", "Tom"]]
print(seats[1])
print(seats[1][2])
print(len(seats))
```

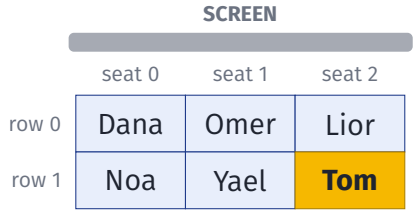


A table is a list of rows

```
seats = [["Dana", "Omer", "Lior"],
         ["Noa", "Yael", "Tom"]]
print(seats[1])
print(seats[1][2])
print(len(seats))
```

OUTPUT

```
['Noa', 'Yael', 'Tom']
Tom
2
```

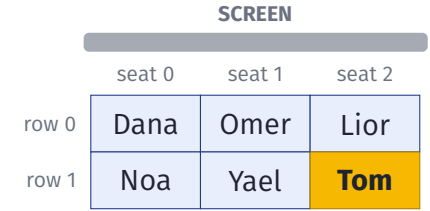


A table is a list of rows

```
seats = [["Dana", "Omer", "Lior"],
         ["Noa", "Yael", "Tom"]]
print(seats[1])
print(seats[1][2])
print(len(seats))
```

OUTPUT

```
['Noa', 'Yael', 'Tom']
Tom
2
```



`seats[1][2]`: row first, then seat.

A loop inside a loop

```
grades = [[90, 85, 70],    # Dana
          [60, 75, 81],    # Omer
          [100, 95, 88]]   # Lior
high = 0
for row in grades:
    for grade in row:
        if grade >= 85:
            high = high + 1
print("Grades of 85+:", high)
```



A loop inside a loop

```
grades = [[90, 85, 70],    # Dana
          [60, 75, 81],    # Omer
          [100, 95, 88]]   # Lior
high = 0
for row in grades:
    for grade in row:
        if grade >= 85:
            high = high + 1
print("Grades of 85+:", high)
```

OUTPUT

```
Grades of 85+: 5
```



A loop inside a loop

```
grades = [[90, 85, 70],    # Dana
          [60, 75, 81],    # Omer
          [100, 95, 88]]   # Lior
high = 0
for row in grades:
    for grade in row:
        if grade >= 85:
            high = high + 1
print("Grades of 85+:", high)
```

OUTPUT

```
Grades of 85+: 5
```

- Outer loop: one **student** (row)
- Inner loop: each **grade** in that row



A list of dictionaries

```
students = [{"name": "Dana", "grade": 92},
            {"name": "Omer", "grade": 58},
            {"name": "Lior", "grade": 77}]
for student in students:
    if student["grade"] >= 60:
        print(student["name"], "passed")
```

A list of dictionaries

```
students = [{"name": "Dana", "grade": 92},
            {"name": "Omer", "grade": 58},
            {"name": "Lior", "grade": 77}]
for student in students:
    if student["grade"] >= 60:
        print(student["name"], "passed")
```

OUTPUT

```
Dana passed
Lior passed
```

A list of dictionaries

```
students = [{"name": "Dana", "grade": 92},
            {"name": "Omer", "grade": 58},
            {"name": "Lior", "grade": 77}]
for student in students:
    if student["grade"] >= 60:
        print(student["name"], "passed")
```

OUTPUT

```
Dana passed
Lior passed
```

Like a spreadsheet: every row has **named** columns.

Why tables? They are everywhere

- 📒 **CSV files**: one line = one row, commas split the columns

Why tables? They are everywhere

- 📒 **CSV files**: one line = one row, commas split the columns
- 📊 **Spreadsheets**: grades, budgets, sales

Why tables? They are everywhere

- 📒 **CSV files**: one line = one row, commas split the columns
- 📊 **Spreadsheets**: grades, budgets, sales
- 📸 **Images**: a grid of pixels, each one a number (Lecture 9)

Why tables? They are everywhere

- 📒 **CSV files**: one line = one row, commas split the columns
- 📊 **Spreadsheets**: grades, budgets, sales
- 📸 **Images**: a grid of pixels, each one a number (Lecture 9)
- 🔢 **Math**: matrices in statistics, physics, economics, AI

Why tables? They are everywhere

- 📒 **CSV files**: one line = one row, commas split the columns
- 📊 **Spreadsheets**: grades, budgets, sales
- 📸 **Images**: a grid of pixels, each one a number (Lecture 9)
- 🔢 **Math**: matrices in statistics, physics, economics, AI
- ♟️ **Games**: a chess board, tic-tac-toe, Minesweeper

Why tables? They are everywhere

- 📒 **CSV files**: one line = one row, commas split the columns
- 📊 **Spreadsheets**: grades, budgets, sales
- 📷 **Images**: a grid of pixels, each one a number (Lecture 9)
- 🔢 **Math**: matrices in statistics, physics, economics, AI
- ♟️ **Games**: a chess board, tic-tac-toe, Minesweeper
- 🚌 **Timetables**: stops × hours

Why tables? They are everywhere

- 📒 **CSV files**: one line = one row, commas split the columns
- 📊 **Spreadsheets**: grades, budgets, sales
- 📷 **Images**: a grid of pixels, each one a number (Lecture 9)
- 🔢 **Math**: matrices in statistics, physics, economics, AI
- ♟️ **Games**: a chess board, tic-tac-toe, Minesweeper
- 🚌 **Timetables**: stops × hours

Rows and columns: the shape of most data in the world.

A matrix: a table with a clear shape

```
m = [[1, 2, 3],
     [4, 5, 6]]
rows = len(m)
cols = len(m[0])
print(rows, "x", cols)
print(m[1][2])
```

A matrix: a table with a clear shape

```
m = [[1, 2, 3],
     [4, 5, 6]]
rows = len(m)
cols = len(m[0])
print(rows, "x", cols)
print(m[1][2])
```

OUTPUT

```
2 x 3
6
```

A matrix: a table with a clear shape

```
m = [[1, 2, 3],
     [4, 5, 6]]
rows = len(m)
cols = len(m[0])
print(rows, "x", cols)
print(m[1][2])
```

OUTPUT

```
2 x 3
6
```

- A **matrix**: every row has the **same length**
- Its shape: **rows** × **columns**
- `m[row][col]`: row first, then column

A matrix: a table with a clear shape

```
m = [[1, 2, 3],
     [4, 5, 6]]
rows = len(m)
cols = len(m[0])
print(rows, "x", cols)
print(m[1][2])
```

OUTPUT

```
2 x 3
6
```

- A **matrix**: every row has the **same length**
- Its shape: **rows** × **columns**
- `m[row][col]`: row first, then column

⚠ Not a matrix

`[[1, 2, 3], [4, 5]]`: row 0 has 3 items, row 1 has 2.

From a CSV file to a list of lists

```
lines = ["Dana,90,85",
         "Omer,60,75",
         "Lior,100,95"]
table = []
for line in lines:
    table.append(line.split(","))
print(table[1])
print(table[2][0], int(table[2][1]) + 5)
```

A CSV file

```
Dana,90,85
Omer,60,75
Lior,100,95
```

From a CSV file to a list of lists

```
lines = ["Dana,90,85",
         "Omer,60,75",
         "Lior,100,95"]
table = []
for line in lines:
    table.append(line.split(","))
print(table[1])
print(table[2][0], int(table[2][1]) + 5)
```

OUTPUT

```
['Omer', '60', '75']
Lior 105
```

A CSV file

```
Dana,90,85
Omer,60,75
Lior,100,95
```

From a CSV file to a list of lists

```
lines = ["Dana,90,85",
         "Omer,60,75",
         "Lior,100,95"]
table = []
for line in lines:
    table.append(line.split(","))
print(table[1])
print(table[2][0], int(table[2][1]) + 5)
```

OUTPUT

```
['Omer', '60', '75']
Lior 105
```

A CSV file

```
Dana,90,85
Omer,60,75
Lior,100,95
```

`split(",")` cuts each line into a row. Everything comes in as **text**: use `int()` before doing math.

Going further: math with tables

- The next slides are an **extra**, for the curious
- Scientists, economists and AI all store numbers in matrices
- Good news: you already know everything you need, **lists and two loops**



Look for the 🚀 **GOING FURTHER** label. Enjoy the ride, no pressure.

Three ways to build the same table

GOING FURTHER

```
grades = [[90, 85], [60, 75]]                  # list of lists: by position
book = {"Dana": [90, 85], "Omer": [60, 75]}    # dictionary: by name
print(grades[1][0], book["Omer"][0])
import numpy as np                             # math object (next lecture!)
arr = np.array(grades)
print(arr.shape, arr[1, 0])
```

Three ways to build the same table

GOING FURTHER

```
grades = [[90, 85], [60, 75]]                  # list of lists: by position
book = {"Dana": [90, 85], "Omer": [60, 75]}    # dictionary: by name
print(grades[1][0], book["Omer"][0])
import numpy as np                             # math object (next lecture!)
arr = np.array(grades)
print(arr.shape, arr[1, 0])
```

OUTPUT

```
60 60
(2, 2) 60
```

Three ways to build the same table

GOING FURTHER

```
grades = [[90, 85], [60, 75]]                  # list of lists: by position
book = {"Dana": [90, 85], "Omer": [60, 75]}    # dictionary: by name
print(grades[1][0], book["Omer"][0])
import numpy as np                             # math object (next lecture!)
arr = np.array(grades)
print(arr.shape, arr[1, 0])
```

OUTPUT

```
60 60
(2, 2) 60
```

- **List of lists**: row number, then column number

Three ways to build the same table

GOING FURTHER

```
grades = [[90, 85], [60, 75]]                  # list of lists: by position
book = {"Dana": [90, 85], "Omer": [60, 75]}    # dictionary: by name
print(grades[1][0], book["Omer"][0])
import numpy as np                             # math object (next lecture!)
arr = np.array(grades)
print(arr.shape, arr[1, 0])
```

OUTPUT

```
60 60
(2, 2) 60
```

- **List of lists**: row number, then column number
- **Dictionary**: a name for each row, easy to read

Three ways to build the same table

GOING FURTHER

```
grades = [[90, 85], [60, 75]]                  # list of lists: by position
book = {"Dana": [90, 85], "Omer": [60, 75]}    # dictionary: by name
print(grades[1][0], book["Omer"][0])
import numpy as np                             # math object (next lecture!)
arr = np.array(grades)
print(arr.shape, arr[1, 0])
```

OUTPUT

```
60 60
(2, 2) 60
```

- **List of lists**: row number, then column number
- **Dictionary**: a name for each row, easy to read
- **NumPy array**: built for math, knows its own shape

Matrix math with two loops: add

GOING FURTHER

```
a = [[1, 2], [3, 4]]
b = [[10, 20], [30, 40]]
total = []
for r in range(len(a)):
    row = []
    for c in range(len(a[0])):
        row.append(a[r][c] + b[r][c])
    total.append(row)
print(total)
```

Matrix math with two loops: add

GOING FURTHER

```
a = [[1, 2], [3, 4]]
b = [[10, 20], [30, 40]]
total = []
for r in range(len(a)):
    row = []
    for c in range(len(a[0])):
        row.append(a[r][c] + b[r][c])
    total.append(row)
print(total)
```

OUTPUT

```
[[11, 22], [33, 44]]
```

Matrix math with two loops: add

GOING FURTHER

```
a = [[1, 2], [3, 4]]
b = [[10, 20], [30, 40]]
total = []
for r in range(len(a)):
    row = []
    for c in range(len(a[0])):
        row.append(a[r][c] + b[r][c])
    total.append(row)
print(total)
```

OUTPUT

```
[[11, 22], [33, 44]]
```

- Same spot, add: `a[r][c] + b[r][c]`
- Both must have the **same shape**

Matrix math with two loops: add

GOING FURTHER

```
a = [[1, 2], [3, 4]]
b = [[10, 20], [30, 40]]
total = []
for r in range(len(a)):
    row = []
    for c in range(len(a[0])):
        row.append(a[r][c] + b[r][c])
    total.append(row)
print(total)
```

OUTPUT

```
[[11, 22], [33, 44]]
```

- Same spot, add: `a[r][c] + b[r][c]`
- Both must have the **same shape**
- **Scale** by 3: use `3 * a[r][c]` instead

Transpose: rows become columns

GOING FURTHER

```
m = [[1, 2, 3],
     [4, 5, 6]]
t = []
for c in range(len(m[0])):
    new_row = []
    for r in range(len(m)):
        new_row.append(m[r][c])
    t.append(new_row)
print(t)
```

Transpose: rows become columns

GOING FURTHER

```
m = [[1, 2, 3],
     [4, 5, 6]]
t = []
for c in range(len(m[0])):
    new_row = []
    for r in range(len(m)):
        new_row.append(m[r][c])
    t.append(new_row)
print(t)
```

OUTPUT

```
[[1, 4], [2, 5], [3, 6]]
```

Transpose: rows become columns

GOING FURTHER

A 2×3 matrix becomes 3×2.

```
m = [[1, 2, 3],
     [4, 5, 6]]
t = []
for c in range(len(m[0])):
    new_row = []
    for r in range(len(m)):
        new_row.append(m[r][c])
    t.append(new_row)
print(t)
```

OUTPUT

```
[[1, 4], [2, 5], [3, 6]]
```

Transpose: rows become columns

GOING FURTHER

```
m = [[1, 2, 3],
     [4, 5, 6]]
t = []
for c in range(len(m[0])):
    new_row = []
    for r in range(len(m)):
        new_row.append(m[r][c])
    t.append(new_row)
print(t)
```

OUTPUT

```
[[1, 4], [2, 5], [3, 6]]
```

A 2×3 matrix becomes 3×2.

Words you will meet

Diagonal: `m[0][0]`, `m[1][1]`, …
Trace: the sum of the diagonal
Identity: 1 on the diagonal, 0 elsewhere

Predict the output: matrices

GOING FURTHER

Predict the output!

```
m = [[2, 0, 1],
     [5, 3, 4]]
print(len(m), len(m[0]))
print(m[1][2] + m[0][0])
total = 0
for i in range(2):
    total = total + m[i][i]
print(total)
```



Predict the output: matrices

GOING FURTHER

Predict the output!

```
m = [[2, 0, 1],
     [5, 3, 4]]
print(len(m), len(m[0]))
print(m[1][2] + m[0][0])
total = 0
for i in range(2):
    total = total + m[i][i]
print(total)
```

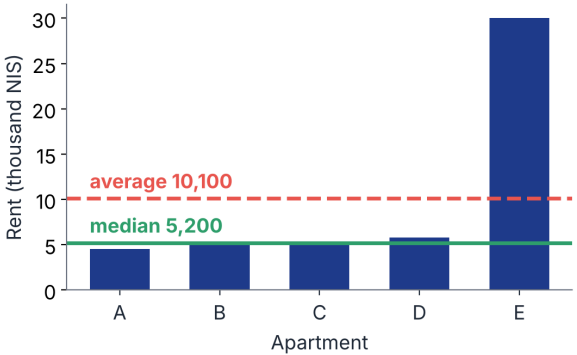
Answer

2 3
6
5

2 rows, 3 columns. `4 + 2 = 6`.
The diagonal: `2 + 3 = 5`.

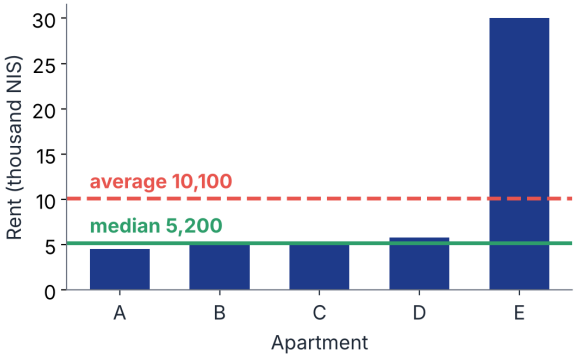


The average can lie



- Five Tel Aviv apartments
- One is a penthouse 🏠

The average can lie



- Five Tel Aviv apartments
- One is a penthouse 🏠
- Average: 10,100 ILS. Nobody pays that!
- **Median**: the middle value, 5,200

The median by hand: sort, take the middle

```
rents = [5800, 4500, 30000, 5200, 5000]
sorted_rents = sorted(rents)
print(sorted_rents)
middle = len(sorted_rents) // 2
print(sorted_rents[middle])
```

The median by hand: sort, take the middle

```
rents = [5800, 4500, 30000, 5200, 5000]
sorted_rents = sorted(rents)
print(sorted_rents)
middle = len(sorted_rents) // 2
print(sorted_rents[middle])
```

OUTPUT

```
[4500, 5000, 5200, 5800, 30000]
5200
```

The median by hand: sort, take the middle

```
rents = [5800, 4500, 30000, 5200, 5000]
sorted_rents = sorted(rents)
print(sorted_rents)
middle = len(sorted_rents) // 2
print(sorted_rents[middle])
```

OUTPUT

```
[4500, 5000, 5200, 5800, 30000]
5200
```

5 items: `5 // 2` is 2, so we take position 2, the middle one.

A median function (odd *and* even)

```
def median(values):
    ordered = sorted(values)
    middle = len(ordered) // 2
    if len(ordered) % 2 == 1:      # odd count
        return ordered[middle]
    else:                          # even count
        return (ordered[middle - 1] + ordered[middle]) / 2
```

```
print(median([5800, 4500, 30000, 5200, 5000]))
print(median([3, 1, 4, 2]))
```

A median function (odd *and* even)

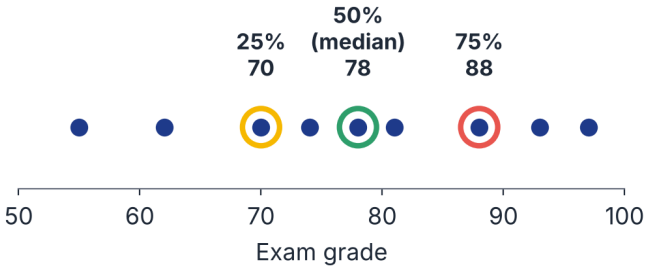
```
def median(values):
    ordered = sorted(values)
    middle = len(ordered) // 2
    if len(ordered) % 2 == 1:      # odd count
        return ordered[middle]
    else:                          # even count
        return (ordered[middle - 1] + ordered[middle]) / 2
```

```
print(median([5800, 4500, 30000, 5200, 5000]))
print(median([3, 1, 4, 2]))
```

OUTPUT

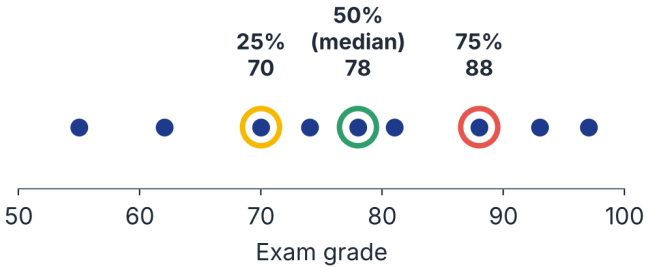
```
5200
2.5
```

Quantiles: where do you stand?



- Sort the grades. The **0.25 quantile**: a quarter are at or below it

Quantiles: where do you stand?



- Sort the grades. The **0.25 quantile**: a quarter are at or below it
- The 0.5 quantile is just the median

A quantile function

```
def quantile(values, q):
    ordered = sorted(values)
    position = int(q * (len(ordered) - 1))
    return ordered[position]

grades = [88, 55, 74, 97, 62, 81, 70, 93, 78]
print(quantile(grades, 0.25))
print(quantile(grades, 0.5))
print(quantile(grades, 0.75))
```

A quantile function

```
def quantile(values, q):
    ordered = sorted(values)
    position = int(q * (len(ordered) - 1))
    return ordered[position]

grades = [88, 55, 74, 97, 62, 81, 70, 93, 78]
print(quantile(grades, 0.25))
print(quantile(grades, 0.5))
print(quantile(grades, 0.75))
```

OUTPUT

```
70
78
88
```

A quantile function

```python
def quantile(values, q):
    ordered = sorted(values)
    position = int(q * (len(ordered) - 1))
    return ordered[position]

grades = [88, 55, 74, 97, 62, 81, 70, 93, 78]
print(quantile(grades, 0.25))
print(quantile(grades, 0.5))
print(quantile(grades, 0.75))
```

OUTPUT

```
70
78
88
```

One simple recipe (it rounds down). NumPy, in L09, has smoother ones.

Try it!

Try it in Colab notebooks/L08/L08_02_tables_and_median.ipynb

1. Build a 3×3 seating chart. Print the middle seat.
2. Count how many grades in a table are below 60
3. Find the median of your last 7 coffee prices





Predict the output

Predict the output!

```
grid = [[1, 2, 3], [4, 5, 6]]
print(grid[1][0])
print(len(grid))
print(sorted([7, 3, 9, 1, 5])[2])
```



Predict the output

Predict the output!

```
grid = [[1, 2, 3], [4, 5, 6]]
print(grid[1][0])
print(len(grid))
print(sorted([7, 3, 9, 1, 5])[2])
```

Answer

4
2
5

`len(grid)` counts rows.
Sorted: `[1, 3, 5, 7, 9]`.



Topic 3

Debugging

Lecture 8 • Dicts & Debugging



TODAY

1. Dictionaries
2. Tables and the middle value
3. **Debugging**
4. Coding with AI

Bugs: three kinds

Kind	What happens
Syntax	The intern can’t read it. Nothing runs.
Runtime	Crashes halfway, with an error message
Logic	Runs fine. Wrong answer. **No message!**



1947: engineers found a real moth stuck inside a Harvard computer.

Joke break: 99 little bugs

99 little bugs in the code,
99 little bugs…

Take one down,
patch it around…

Joke break: 99 little bugs

99 little bugs in the code,
99 little bugs…

Take one down,
patch it around…

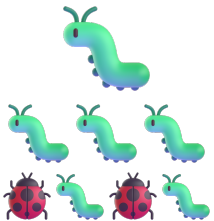
127 little bugs in the code.

Joke break: 99 little bugs

99 little bugs in the code,
99 little bugs…

Take one down,
patch it around…

127 little bugs in the code.



Fix **one** bug at a time, and run the code after each fix.

Read the error from the bottom up

average.py

```
def average(values):
    return sum(values) / len(values)
grades = []
print(average(grades))
```

Colab draws it a little differently. Read it the same way.

ERROR

```
Traceback (most recent call last):
  File "average.py", line 4, in <module>
    print(average(grades))
  File "average.py", line 2, in average
    return sum(values) / len(values)
ZeroDivisionError: division by zero
```

Read the error from the bottom up

average.py

```
def average(values):
    return sum(values) / len(values)
grades = []
print(average(grades))
```

1. **Last line**: what went wrong

Colab draws it a little differently. Read it the same way.

ERROR

```
Traceback (most recent call last):
  File "average.py", line 4, in <module>
    print(average(grades))
  File "average.py", line 2, in average
    return sum(values) / len(values)
ZeroDivisionError: division by zero
```

Read the error from the bottom up

average.py

```
def average(values):
    return sum(values) / len(values)
grades = []
print(average(grades))
```

1. **Last line**: what went wrong
2. **Above it**: where (line 2)

Colab draws it a little differently. Read it the same way.

ERROR

```
Traceback (most recent call last):
  File "average.py", line 4, in <module>
    print(average(grades))
  File "average.py", line 2, in average
    return sum(values) / len(values)
ZeroDivisionError: division by zero
```

Read the error from the bottom up

average.py

```
def average(values):
    return sum(values) / len(values)
grades = []
print(average(grades))
```

1. **Last line**: what went wrong
2. **Above it**: where (line 2)
3. **Top**: who called it (line 4)

Colab draws it a little differently. Read it the same way.

ERROR

```
Traceback (most recent call last):
  File "average.py", line 4, in <module>
    print(average(grades))
  File "average.py", line 2, in average
    return sum(values) / len(values)
ZeroDivisionError: division by zero
```

Errors you will meet

Error	In plain words	Typical cause
`NameError`	“Never heard of that name”	a typo: `print(nme)`
`TypeError`	“These types don’t mix”	`"age: " + 20`
`ValueError`	“Right type, strange value”	`int("twenty")`
`IndexError`	“No item at that position”	`[1, 2, 3][3]`
`KeyError`	“No such key”	`menu["tea"]`
`ZeroDivisionError`	“Can’t divide by zero”	average of an empty list



The intern is confused. The last line says why.

A logic bug: no error, wrong answer

```
def total_price(prices):
    for price in prices:
        total = 0
        total = total + price
    return total

print(total_price([10, 20, 30]))
```

A logic bug: no error, wrong answer

We expected 60!

```
def total_price(prices):
    for price in prices:
        total = 0
        total = total + price
    return total

print(total_price([10, 20, 30]))
```

OUTPUT

```
30
```

A logic bug: no error, wrong answer

```
def total_price(prices):
    for price in prices:
        total = 0
        total = total + price
    return total

print(total_price([10, 20, 30]))
```

OUTPUT

```
30
```

We expected 60!



No message. Where is the bug?

Print debugging: make the intern talk

```
def total_price(prices):
    for price in prices:
        total = 0
        total = total + price
        print("total is", total)
    return total
print(total_price([10, 20, 30]))
```

Print debugging: make the intern talk

```
def total_price(prices):
    for price in prices:
        total = 0
        total = total + price
        print("total is", total)
    return total
print(total_price([10, 20, 30]))
```

OUTPUT

```
total is 10
total is 20
total is 30
30
```

Print debugging: make the intern talk

```python
def total_price(prices):
    for price in prices:
        total = 0
        total = total + price
        print("total is", total)
    return total
print(total_price([10, 20, 30]))
```

OUTPUT

```
total is 10
total is 20
total is 30
30
```

Found it! `total` goes back to 0 every round.

Fix: move `total = 0` above the loop. Now the output is `60`.

Your debugging checklist

1. Read the **last line** of the error





Your debugging checklist

1. Read the **last line** of the error
2. Go to the **line number** it names





Your debugging checklist

1. Read the **last line** of the error
2. Go to the **line number** it names
3. **Print** the values just before that line





Your debugging checklist

1. Read the **last line** of the error
2. Go to the **line number** it names
3. **Print** the values just before that line
4. Explain the code out loud, line by line

Explaining to a teddy bear counts. Really.

Next level: a real step-by-step debugger, in Lecture 10.





Crash or catch?

Remember the Bug Zoo from Lecture 3?

```
hours = int(input("Hours? "))
```

The user typed `3.5`

ERROR

```
ValueError: invalid literal for int()
with base 10: '3.5'
```

Crash or catch?

Remember the Bug Zoo from Lecture 3?

```
hours = int(input("Hours? "))
```

The user typed `3.5`

ERROR

```
ValueError: invalid literal for int()
with base 10: '3.5'
```

- One typo, and the **whole program dies**

Crash or catch?

Remember the Bug Zoo from Lecture 3?

```
hours = int(input("Hours? "))
```

The user typed `3.5`

ERROR

```
ValueError: invalid literal for int()
with base 10: '3.5'
```

- One typo, and the **whole program dies**
- A real app doesn't crash. It says *"please try again"*

Crash or catch?

Remember the Bug Zoo from Lecture 3?

```
hours = int(input("Hours? "))
```

The user typed `3.5`

ERROR

```
ValueError: invalid literal for int()
with base 10: '3.5'
```

- One typo, and the **whole program dies**
- A real app doesn't crash. It says *"please try again"*
- Python lets you **catch** an error: `try` / `except`



Errors are not only for reading. You can plan for them.

`try` / `except`: the syntax

```
text = input("Your age? ")
try:
    age = int(text)
    print(f"Next year: {age + 1}")
except ValueError:
    print("Please type a whole number")
```

- `try`: “Intern, **try** this”

`try` / `except`: the syntax

```
text = input("Your age? ")
try:
    age = int(text)
    print(f"Next year: {age + 1}")
except ValueError:
    print("Please type a whole number")
```

OUTPUT

```
Your age? 20
Next year: 21
```

- `try`: “Intern, **try** this”

`try` / `except`: the syntax

```
text = input("Your age? ")
try:
    age = int(text)
    print(f"Next year: {age + 1}")
except ValueError:
    print("Please type a whole number")
```

- `try`: “Intern, **try** this”
- Error inside `try`? Jump to `except`. The rest of `try` is **skipped**

OUTPUT

```
Your age? 20
Next year: 21
```

OUTPUT

```
Your age? abc
Please type a whole number
```

`try` / `except`: the syntax

```
text = input("Your age? ")
try:
    age = int(text)
    print(f"Next year: {age + 1}")
except ValueError:
    print("Please type a whole number")
```

- `try`: “Intern, **try** this”
- Error inside `try`? Jump to `except`. The rest of `try` is **skipped**
- Same colon and indent as `if`

OUTPUT

```
Your age? 20
Next year: 21
```

OUTPUT

```
Your age? abc
Please type a whole number
```

Name the error you catch

The pizza bill from Lecture 3, now safe:

```
pizza_price = 160
try:
    num_friends = int(input("Friends? "))
    each = pizza_price / num_friends
    print(f"Each pays {each:.2f} ILS")
except ValueError:
    print("Type a whole number")
except ZeroDivisionError:
    print("No friends? You pay!")
```

Name the error you catch

The pizza bill from Lecture 3, now safe:

```
pizza_price = 160
try:
    num_friends = int(input("Friends? "))
    each = pizza_price / num_friends
    print(f"Each pays {each:.2f} ILS")
except ValueError:
    print("Type a whole number")
except ZeroDivisionError:
    print("No friends? You pay!")
```

OUTPUT

```
Friends? 3
Each pays 53.33 ILS
```

OUTPUT

```
Friends? three
Type a whole number
```

OUTPUT

```
Friends? 0
No friends? You pay!
```

Name the error you catch

The pizza bill from Lecture 3, now safe:

```
pizza_price = 160
try:
    num_friends = int(input("Friends? "))
    each = pizza_price / num_friends
    print(f"Each pays {each:.2f} ILS")
except ValueError:
    print("Type a whole number")
except ZeroDivisionError:
    print("No friends? You pay!")
```

OUTPUT

```
Friends? 3
Each pays 53.33 ILS
```

OUTPUT

```
Friends? three
Type a whole number
```

OUTPUT

```
Friends? 0
No friends? You pay!
```

One **except** per error you expect. Its name: the last line of the error.

Ask until it's valid

```
while True:
    try:
        age = int(input("Age? "))
        break
    except ValueError:
        print("Try again!")
print(f"Thanks! Age: {age}")
```

Ask until it's valid

```
while True:
    try:
        age = int(input("Age? "))
        break
    except ValueError:
        print("Try again!")
print(f"Thanks! Age: {age}")
```

OUTPUT

```
Age? abc
Try again!
Age? twenty
Try again!
Age? 20
Thanks! Age: 20
```

Ask until it's valid

```
while True:
    try:
        age = int(input("Age? "))
        break
    except ValueError:
        print("Try again!")
print(f"Thanks! Age: {age}")
```

OUTPUT

```
Age? abc
Try again!
Age? twenty
Try again!
Age? 20
Thanks! Age: 20
```

`while True` repeats forever. `break` (Lecture 6) runs only when `int(...)` worked. Bad input? `break` is skipped, and we ask again.

Predict the output: `try` / `except`

Predict the output!

```
for text in ["7", "seven", "0"]:
    try:
        print(14 / int(text))
    except ValueError:
        print("not a number")
    except ZeroDivisionError:
        print("zero!")
```

Predict the output: `try` / `except`

Predict the output!

```
for text in ["7", "seven", "0"]:
    try:
        print(14 / int(text))
    except ValueError:
        print("not a number")
    except ZeroDivisionError:
        print("zero!")
```

Answer

```
2.0
not a number
zero!
```

`int("seven")` fails first: `ValueError`.
`int("0")` works, then `14 / 0` fails: `ZeroDivisionError`.

Colab's Gemini can explain errors

- Look for the **Gemini option shown under the error**
- It explains and suggests a fix

Colab’s Gemini can explain errors

- Look for the **Gemini option shown under the error**
- It explains and suggests a fix
- Read the explanation **first**

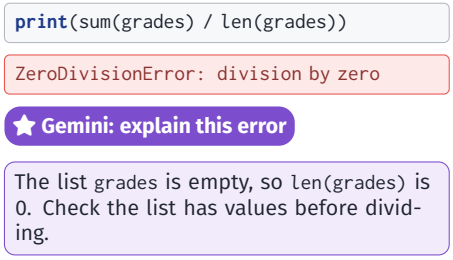


Illustration. The real button looks a bit different.

Colab's Gemini can explain errors

- Look for the **Gemini option shown under the error**
- It explains and suggests a fix
- Read the explanation **first**

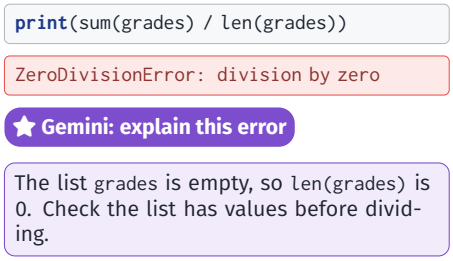


Illustration. The real button looks a bit different.

Privacy

Gemini may keep what you send for up to 18 months. No personal data!

Source: Google Colab FAQ. Button names change, so look for "Gemini".

Try it!

Try it in Colab notebooks/L08/L08_03_debugging.ipynb

1. Three broken cells: read each error from the bottom up
2. Fix them, one at a time
3. Cell 4 has a logic bug: add `print`s to find it
4. Make the age question safe: ask until the input is a number



Predict the output (and the error!)

Predict the output!

```
seats = [["Dana", "Omer"],
         ["Noa", "Yael"]]
print(seats[1][1])
print(seats[0][2])
```



Predict the output (and the error!)

Predict the output!

```
seats = [["Dana", "Omer"],
         ["Noa", "Yael"]]
print(seats[1][1])
print(seats[0][2])
```

Answer

```
Yael
IndexError: list index out
of range
```

Each row has seats 0 and 1 only. The first line still runs.



Topic 4

Coding with AI

Lecture 8 • Dicts & Debugging



TODAY

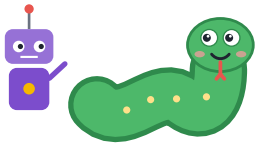
1. Dictionaries
2. Tables and the middle value
3. Debugging
4. **Coding with AI**

Chat assistants: what they are

- Examples: ChatGPT, Claude, Gemini
- They predict the **next word**, again and again (L01!)

Chat assistants: what they are

- Examples: ChatGPT, Claude, Gemini
- They predict the **next word**, again and again (L01!)
- Learned from huge amounts of text and code
- They sound sure, even when guessing



Good at, bad at

👍 Good at

- Explaining errors in plain words
- Short, common pieces of code
- Examples and practice questions

Good at, bad at

Good at

- Explaining errors in plain words
- Short, common pieces of code
- Examples and practice questions

Bad at

- Being right every time
- Saying “I don’t know”
- Knowing your course rules

Good at, bad at

Good at

- Explaining errors in plain words
- Short, common pieces of code
- Examples and practice questions

Bad at

- Being right every time
- Saying “I don’t know”
- Knowing your course rules

A helpful tutor that sometimes makes things up.

Confidently wrong



"Here is a median function!"

```
def median(values):
    middle = len(values) // 2
    return values[middle]

print(median([5, 1, 9]))
```

Confidently wrong

"Here is a median function!"

```
def median(values):
    middle = len(values) // 2
    return values[middle]

print(median([5, 1, 9]))
```

OUTPUT

```
1
```

The median of 5, 1, 9 is 5!

Confidently wrong

"Here is a median function!"

```
def median(values):
    middle = len(values) // 2
    return values[middle]

print(median([5, 1, 9]))
```

OUTPUT

```
1
```

The median of 5, 1, 9 is 5!



It forgot to **sort**. Looks right, is wrong.

Privacy: what never goes into a chatbot

- Names, grades or ID numbers of real people
- Passwords and private messages
- Your own personal or medical details
- Data your workplace asked you to protect



Instead

Make up fake data with the same shape: `"Student A": 85`

A good prompt, in four parts

🎯

1. Goal

What should the code do?

A good prompt, in four parts



1. Goal

What should the code do?



2. Example

Input → expected output

A good prompt, in four parts



1. Goal

What should the code do?



2. Example

Input → expected output



3. Limits

Only what we learned

A good prompt, in four parts



1. Goal

What should the code do?



2. Example

Input → expected output



3. Limits

Only what we learned



4. Explain

Ask why, line by line

A good prompt, in four parts



1. Goal

What should the code do?



2. Example

Input → expected output



3. Limits

Only what we learned



4. Explain

Ask why, line by line

Not quite right? Reply and refine.

A weak prompt vs a good prompt

⚠ Weak

“python word counter”

A weak prompt vs a good prompt

⚠ Weak

“python word counter”

💬 Good

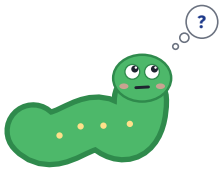
“I am a beginner in Python. Write a function `count_words(words)` that gets a list of words and returns a dictionary of counts. Example: `['hi', 'yo', 'hi']` → `{'hi': 2, 'yo': 1}`. Use only `for` loops and dictionaries. Explain each line simply.”

Goal + example + limits + explain

Explain, don’t fix

💬 A good prompt

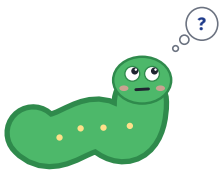
“Here is my code and the error. Explain what the error means and which line causes it. **Do not** give me the fixed code.”



Explain, don’t fix

💬 A good prompt

“Here is my code and the error. Explain what the error means and which line causes it. **Do not** give me the fixed code.”



- You fix it yourself, so you learn the pattern
- Next time, no AI needed

Always test AI code

```
print(median([5, 1, 9]), "expected 5")
print(median([4, 1, 3, 2]), "expected 2.5")
print(median([7]), "expected 7")
```

Tested: the AI's median from the "Confidently wrong" slide

Always test AI code

```
print(median([5, 1, 9]), "expected 5")
print(median([4, 1, 3, 2]), "expected 2.5")
print(median([7]), "expected 7")
```

OUTPUT

```
1 expected 5
3 expected 2.5
7 expected 7
```

Tested: the AI's median from the "Confidently wrong" slide

Always test AI code

```
print(median([5, 1, 9]), "expected 5")
print(median([4, 1, 3, 2]), "expected 2.5")
print(median([7]), "expected 7")
```

OUTPUT

```
1 expected 5
3 expected 2.5
7 expected 7
```

- Use cases you can solve by hand
- Add tricky ones: even count, one item

Tested: the AI's median from the "Confidently wrong" slide

Academic integrity

- AI is your **tutor**, not your ghostwriter
- You must be able to explain every line you hand in
- Say when and how you used AI



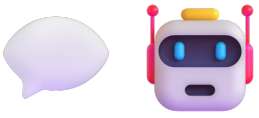
⚠ Course AI policy

- **OK:** ask AI to explain ideas, errors, examples
- **Not OK:** hand in code you cannot explain line by line
- Used AI in homework? Say how in a comment at the top
- The final exam is **without** AI

Try it!

Try it in Colab notebooks/L08/L08_04_prompting.ipynb

1. Write a 4-part prompt for `most_common(votes)`
2. Paste the answer into Colab and test it on 3 cases
3. Ask the AI to explain one line you don't understand



Predict the output: the AI wrote this

Passing grade: 60 and up

```
def is_passing(grade):
    if grade > 60:
        return True
    else:
        return False
print(is_passing(75))
print(is_passing(60))
```

Predict the output: the AI wrote this

Passing grade: 60 and up

```
def is_passing(grade):
    if grade > 60:
        return True
    else:
        return False
print(is_passing(75))
print(is_passing(60))
```

Answer

True
False

A 60 should pass! The fix: >=. Always test the edge.

Bug Zoo: the missing key

BUG ZOO

```
menu = {"falafel": 18, "tea": 8}
print(menu["Falafel"])
```

ERROR

```
KeyError: 'Falafel'
```

Keys must match exactly

`"Falafel"` and `"falafel"` are different keys.

Fix: `menu["falafel"]`, or check first with `in`

Bug Zoo: loops and tables

BUG ZOO

```
menu = {"tea": 8, "coffee": 12}
for item, price in menu:
    print(item, price)
```

ERROR

```
ValueError: too many values to
unpack (expected 2)
```

Fix: loop over `menu.items()`

```
seats = [["Dana", "Omer"],
         ["Noa", "Yael"]]
print(seats[2][0])
```

ERROR

```
IndexError: list index out of
range
```

Fix: the rows are only 0 and 1

Bug Zoo: the `except` that catches everything

BUG ZOO

```
try:
    price = int(input("Price? "))
    print(f"With VAT: {prise * 1.18}")
except:
    print("Please type a number")
```

User types `100`, a fine number:

OUTPUT

```
Price? 100
Please type a number
```

It hides the real bug

A bare `except:` catches **every** error, even our typo `prise`. It blames the user!

Fix: name the error:

```
except ValueError:
```

Now the real bug shows:

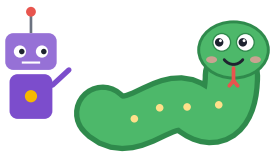
```
NameError
```

AI Corner: quiz me on dictionaries

AI CORNER

A good prompt

"Ask me 3 short 'predict the output' questions about Python dictionaries. Show each answer only after I reply."



Double-check

Run each question in Colab. Is the AI's answer key right?

Recap

✓ Today you learned

- Dictionaries map keys to values: look up, count, loop
- Tables are lists of rows. Median = middle of the sorted list
- Read errors bottom-up, print values, test AI code
- `try` / `except ValueError`: catch an error instead of crashing



Next time: objects and Python's power tools

- Build your own type: a `Point`, a `Cell`, a bank account
- NumPy: the median of a million numbers, in one line
- A photo is just a table of numbers
- Pandas: real data tables, like a spreadsheet





3D emoji: Microsoft Fluent Emoji (MIT License). Pip the snake: original drawing. Full list: images/CREDITS.md